Computational Programming with Python

Lecture 1: First steps - A bit of everything

Center for Mathematical Sciences, Lund University

Lecturer: `Robert Klöfkorn` (Malin Christersson)

This lecture

- About this course
- About Python
- Data types and variables
- Conditional expressions
- Loops
- Importing modules
- Presentation of teaching assistants

About this course 1/2

14 Lectures:

- 5 basic lectures (basic data types, for loops, plotting, functions, etc.)
- 2 lectures on matrix/vector and linear algebra
- 2 lectures on object oriented programming
- 1 lecture on file handling (data I/O)
- 2 lectures on advanced plotting including interactive plotting
- 1 lecture debugging, timing, and time complexity
- 1 lecture on data analysis with pandas

About this course 2/2

- Lectures are followed by training exercises (**hand-in possible**)
- Homework 1 and 2 (oral presentation 15-20min)
- Final projects (oral presentation 20-40m)

Need to pass HW1 & HW2, the Quiz, and the Final Project to pass the course.

Course Book

Scientific Computing with Python
Claus Führer, Jan Erik Solem, Olivier Verdier (2nd ed.), 2021

www.packtpub.com
Available as **eBook** in the LU library.

What do we expect from you?

- Engagement in the course, i.e. attendance in lectures and training exercises
- Work with the material (only that will make you learn programming)
- Read the corresponding book chapter before the lecture
- If you cannot make it to the lecture (don't email me) it is expected that you keep up with the learning material by yourself

`Comment from student:` *I would have appreciated it if the lectures took a bit more of a middle ground. I was very lucky that my class was so nice. Often some of my classmates who have programmed before would explain more thoroughly the stuff we went through in the lectures to me.*

Questions?

Why Python?

Python is...

- Free and open source
- It is a *scripting language* - an interpreted (not compiled) language
- It has plenty of libraries among others the scientific ones: linear algebra; visualisation tools; plotting; image analysis; differential equations solving; symbolic computations; statistics; etc.
- It's used everywhere: Scientific computing, scripting, web sites, text parsing, data mining, ...

The right tool for the right problem

Python is far from perfect.

But for what we want to do here it is very suitable, because

- it is relatively easy to learn
- suitable for (smaller) computation in terms of speed
- one of the most popular programing languages right now

It provides all that we want to teach in this course and for what you need during your studies at LU.

Why should you take this course very serious?

Many courses in the Math and Physics education include programming projects at all stages of the education. In Physics roughly 30% of the total credits are obtained through solving computational tasks. Most of these tasks involve analyzing experimental data gathered during the labs.

- `MATB21` Project: Develop functions to perform (numerical) integration, optimization, and Taylor expansion of various multivariable functions. Test different visualization methods.
- `MATB22` Project: A selection of problems that are to be solved numerically.
- `NUMA41` Project: Solve an ODE numerically and other training exercises
- `NUMB11` Several programming projects
- `FYSB21` Simulation: Develop a program (OOP) that simulates how heat dissipates through different materials. Simulate an earth-asteroid impact. This project relies heavily on vectorization.
- `FYSB22` Project: Implement functions to solve the time-independent Schrödinger Equation for arbitrary potentials. This requires the use of bisection searches to solve for the boundary conditions.
- `FYSB23` Simulation: Develop a program that simulates a non-linear (chaotic)

system, investigate the partition of energy (related to the Fermi-Pasta-Ulam-Tsingou problem). This requires careful thought about time complexity.

- `FYSB23` Project: Use two different computational models (self-avoiding random walk and Monte Carlo simulation) to investigate statistical properties related to protein folding.
- `FYSB24` Project: Perform very detailed spectroscopic analysis using several different methods (using Python).

And many more...

Premisses

- We work with Python version ≥ 3.x
- We use an `IPython` shell
- We use the work environment `Spyder` (feel free to use any other IDE of your choice)
- We work (later) with the IPython notebook
- We start our programs with

In [1]:

```
from numpy import *
# will be explained later
print("Hello")
```

```
Hello
```

Examples

Python may be used in *interactive mode*
(executed in an IPython shell)

In [2]:

```
x = 3
```

In [3]:

```
y = 5
```

In [5]:

```
print(x + y)
```

```
8
```

Note:
`In [2]` is the prompt in IPython shell. It counts the statements. In the more basic Python shell the prompt string is `>>>`

IPython

IPython is an enhanced Python interpreter.

- Use the arrow keys to visit previous commands.
- Use the `TAB` key to auto-complete.
- To get help on an object just type `?` after it and then return.

Examples: Linear Algebra

Let us solve

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \cdot x = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

In [6]:

```
from numpy import array
from scipy.linalg import solve
M = array([[1., 2.],
           [3., 4.]])
V = array([2., 1.])
x = solve(M, V)
print(x)
```

```
[-3.   2.5]
```

Note: A line can be continued on the next line without any continuation symbol as long as parentheses or brackets are not closed.

More examples

Computing $e^{i\pi}$ and 2^{100}:

In [7]:

```
from numpy import *
print(exp(1j * pi)) # should return -1
```

```
(-1+1.2246467991473532e-16j)
```

In [8]:

```
print(2**100)
# x = 4
```

```
1267650600228229401496703205376
```

Note: Everything following `#` is treated as a comment

Data types and variables

Numerical data types

In Python we call a number

- $i \in \mathbb{Z}$ an *integer* (`int`), and
- $x \in \mathbb{R}$ a *real number* (`float`) and,
- $z \in \mathbb{C}$ a *complex number* (`complex`).

Arithmetic operators

`+ - * /` (addition, subtraction, multiplication, division)
`**` (exponentiation or power)
`%` (modulus operator, yields remainder)
`//` (floor division, yields quotient)

In [9]:

```
2**(2+2)
```

Out[9]:

```
16
```

In [10]:

```
1j**2
```

Out[10]:

```
(-1+0j)
```

Variables

A variable is a reference to an object. One uses the **assignment operator** = to assign a value to a variable.

In [11]:

```
my_int = 42
k = 5
a = 6.5
b = 4.5
j = 3
my_float = 3.14
my_complex = 5+1j
z = 5 + j
print("z = ",z)
my_sum = my_float+my_complex
print(my_sum)
print(type(my_sum))
print(type(k))

my_int2 = int(3.14)
my_int3 = int(3.76)
print(my_int2)
print(my_int3)
```

```
z =  8
(8.14+1j)
<class 'complex'>
<class 'int'>
3
3
```

For complex numbers, `j` is a suffix to the imaginary part. The code `5+j` will not work.

Strings

Strings are `lists` of characters enclosed by single or double quotes.

```
str1 = '"Eureka" he shouted.'
str2 = "He had found what is now known as Archimedes'
principle."
```

Multiple lines

You may also use triple quotes for strings including multiple lines:

In [12]:

```
str3 = """This "Hello" is
a long,
long string."""
print(str3)
```

```
This "Hello" is
a long,
long string.
```

Conditional expressions

Book p. 20, 22, 37

Definition

A *conditional expression* is an expression that may have the *Boolean* value `True` or `False`.

In [13]:

```
x = True
y = False
print(type(x))
print(x)
```

```
<class 'bool'>
True
```

Comparison operators

Comparison operators act on numerical operands and yield a Boolean value (`True` or `False`).

`==`	(equal to)
`!=`	(not equal to)
`<`	(less than)
`>`	(greater than)
`<=`	(less than or equal to)
`>=`	(greater than or equal to)

Boolean variables

We can assign Boolean values to variables in different ways.

In [14]:

```
a = True
b = 0 < 3 < 6
c = (3*15 == 10)
print("b =", b)
print("c =", c)
print(type(c))
```

```
b = True
c = False
<class 'bool'>
```

Boolean operators

Boolean operators (`and`, `or`, `not`) act on Boolean operands and yield a Boolean value.

In [15]:

```
print(not 3 > 5)
print(36 % 6 == 0 and 36 < 0)
print(36 % 6 == 0 or 36 < 0)
```

```
True
False
True
```

Arithmetic operators (again)

`+ - * /` (addition, subtraction, multiplication, division)

`**` (exponentiation)

`%` (modulus operator, yields remainder)

`//` (floor division, yields quotient)

In [16]:

```
from numpy import ceil
print("Using integer division to divide 20 by 6:")
print("the quotient is", 20//6)
print("the remainder is", 20%6)
print("the devision is ", 20/6)
print("the devision is ", int(20/6))
print("the devision is ", int(23/6))
print("the devision rounded up is ", int(ceil(23/6)))
```

```
Using integer division to divide 20 by 6:
the quotient is 3
the remainder is 2
the devision is  3.3333333333333335
the devision is  3
the devision is  3
the devision rounded up is  4
```

Conditional expressions examples

In [17]:

```
3 < 4 == 1
```

Out[17]:

```
False
```

In [18]:

```
a = False
2 != 3 < 4 or a
```

Out[18]:

```
True
```

In [19]:

```
print(not False or True and False)

print(False and (True or not False))
```

```
True
False
```

The priority of Boolean operators can be compared with arithmetic operators, addition (→ `or`), multiplication (→ `and`), and sign change (→ `not`).

Conditional statements

`if` statement

A conditional statememt delimits a *block* that will be executed if the condition is `True`. An *optional* block, started with the keyword `else` will be executed if the condition is `False`.

Example

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{else} \end{cases}$$

In [20]:

```
x = -5
if x >= 0:
    print(x)
else:
    print(-x)
```

```
5
```

Colon marks the beginning of a block.

```
if a < b:
    s=a/b
else:
    s=2*b-a
```

Indentation is **mandatory**and shows the extent of the block.

The block ends when the indentation returns to the previous level.

One branch

In [24]:

```
age = 19
if age%2 == 0:
    print("It's an even number")
```

Two branches

In [25]:

```
if age%2 == 0:
    print("It's an even number")
else:
    print("It's an odd number")
```

```
It's an odd number
```

Several branches

In [26]:

```python
age = 6
if age < 18:
    print("You are a child.")
elif age < 68:
    print("You are an adult.")
else:
    print("You may retire.")
```

```
You are a child.
```

Error messages

By using Boolean expressions you can check for potential errors:

In [29]:

```
denom = 0
num = 3

if not denom == 0:
    frac = num/denom
else:
    # later we learn how to do this right using Exceptions
    print("Don't divide by zero!")
```

```
Don't divide by zero!
```

Loops

You can iterate in Python by using a `while` loop

In [31]:

```
while some_condition:
    # code that is repeated
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[31], line 1
----> 1 while some_condition:
      2     # code that is repeated
      3     pass

NameError: name 'some_condition' is not defined
```

or a `for` loop

In [ ]:

```python
for some_variable in some_set_of_values:
    # code that is repeated
```

`while` loops

Example: Find

$$\sum_{i=0}^{10} 1$$

In [32]:

```
i = 0
while i <= 10: # some_condition
    # task that is repeated
    print(i)
    # i = i + 1
    i += 1

print("We obtain i =",i)
```

```
0
1
2
3
4
5
6
7
8
9
10
We obtain i = 11
```

What will happen if you forget to write the last row of the `while` loop?

Another example

In [33]:

```
i = 0
x = 1

# compute 2**10
while i < 10:
    x = x * 2
    i = i + 1 # increment counter i

print("We got     ", x)
print("We expected",2**10)
```

```
We got      1024
We expected 1024
```

Note - Infinite loops

```
# Careful with the condition statement here!
while True:
    # repeat task until forever
    print("Hello")
```

Repeating a task

One typical use of the `for` loop is to repeat a certain task a fixed number of times:

`for` loops

Example: Find

$$\sum_{i=0}^{10} 1$$

In [34]:

```
s = 0
for i in range(11): # note the number 11
    s = s + 1

print("We obtain s =",s)
```

```
We obtain s = 11
```

Indentation

The part to be repeated in the for loop has to be properly *indented*.

```
for elt in my_list:
    do_something()
    something_else()
    etc
print("loop finished") # outside of the for loop
```

In contrast to other programming languages, the indentation in Python is **mandatory**.
Many other kinds of Python structures also have to be indented and this will be covered when introduced.

The `range` function

You can use:

`range(stop)` a sequence of integers ≥ `0` and < `stop`

`range(start, stop)` a sequence of integers ≥ `start` and < `stop`

`range(start, stop, step)` a sequence of integers that starts with `start` and then increases by `step` as long as the number is < `stop`.

Example: Show all non-negative multiples of 15 that are less than 100.

In [35]:

```
for i in range(0, 100, 15):
    print(i)
```

```
0
15
30
45
60
75
90
```

Importing modules

Importing Numpy

In order to use standard mathematical functions in Python, you must import some module. We will use `numpy` exclusively.
You can import numpy and then use dot-notation after the name `numpy`:

In [ ]:

```
import numpy
print(numpy.exp(1j*numpy.pi))
```

You can import numpy and give it a shorter name:

In [ ]:

```
import numpy as np
print(np.exp(1j*np.pi))
```

More about imports

You can choose what you want to import to avoid dot-notation:

In [36]:

```
from numpy import exp, pi

print(exp(1j*pi))
```

```
(-1+1.2246467991473532e-16j)
```

You can make a *star import* to import everything from a module:

In [ ]:

```
from numpy import *
print(exp(1j*pi))
```

Configuration of Spyder

Find the `Preferences`:
`Tools - Preferences` (Windows/Linux)
`Python - Preferences` (Mac)
Choose `Editor` and the tab `Advanced settings`, then click `Edit template for new modules`. A file with the name `template.py` will be opened where you can enter the code for imports that will be included in all new files.
See instructions on Canvas for more information.

How to run a piece of code

There are two phases in Python programming:

1. Writing the code
2. Executing (running) it For Task 2, one needs to give the code to the Python interpreter which reads the code and figures out what to do with it.

Short snippets of code can be written directly in the interpreter and executed interactively. We use the interpreter `IPython` which has extra features.

For writing a larger program, one generally uses a text editor which can highlight code in a good way.

There are also development suites which bundle the interpreter, editor and other things into the same program.